

Lab 3

System Call vs APIs

TA: Ruei-Lin Gao

Prof.: Hsung-Pin Chang

NCHU System & Network Lab

$ 類 UNIX 系統

- 類 Unix 系統（Unix-like systems）是指一系列在設計理念、使用者介面、核心功能等方面與 Unix 作業系統相似的電腦作業系統。
- 常見的類 Unix 系統包括：
  - Linux: 最受歡迎的類 Unix 系統，有眾多發行版（如 Ubuntu, Fedora, Debian, CentOS 等），廣泛應用於伺服器、嵌入式系統、個人電腦等 (特別注意：Linux 本身其實指的是 Linux kernel)
  - macOS: 蘋果公司的桌面作業系統
  - BSD 系列：包括 FreeBSD, OpenBSD, NetBSD 等，它們是從原始 Unix 程式碼發展而來的。

$ 類 UNIX 系統

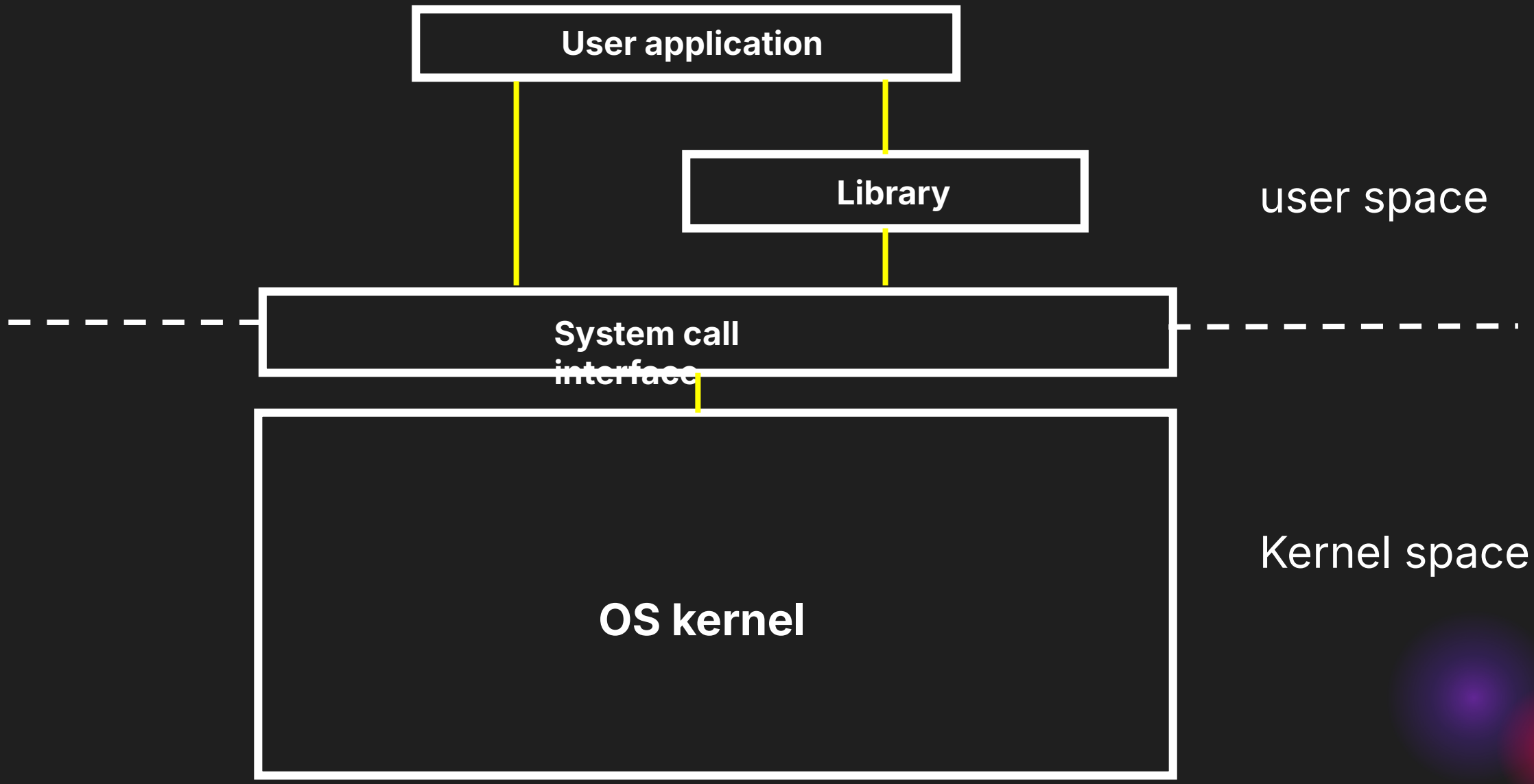
類 UNIX 系統特色

- 一切都是檔案：類 Unix 系統把很多東西（像硬碟、終端機、設定、程序）都當作檔案來處理，讓操作方式統一、簡單。
- 命令列操作（CLI）：可以用文字指令控制系統，指令能組合使用，適合做自動化工作。
- 多使用者、多工：可以讓多個人同時登入，也能同時跑很多程式，效率高又安全。
- 行程管理：每個正在執行的程式叫做「process」，系統會分配資源並確保它們正常運作。
- Shell 介面：Shell 是用來輸入指令的工具，像 Bash 或 Zsh ，是使用者和系統之間的橋樑。
- 開放與相容：很多類 Unix 系統（像 Linux 、 FreeBSD ）是開放原始碼的，遵守 POSIX 標準，讓軟體可以在不同系統上運作。

$ POSIX

- POSIX （Portable Operating System Interface）是一組由 IEEE 制定的標準
- 用來規範 UNIX 及類 UNIX 系統對外提供的程式介面與行為
- 讓應用程式可以在不同平台（如 macOS、 BSD ，以及以 Linux 為核心的系統，例如 Ubuntu、 Debian、 Red Hat 等）之間更容易移植。

$ POSIX


User application
Library
System call interface
OS kernel
user space
Kernel space

$ POSIX

- POSIX 規範內容

  1. System interfaces （常見的 System Call) – OS 要提供應用程式使用 OS 服務的介面例如：read() 、 write() 、 fork() 、 wait()

  2. Library(通常封裝 system call ，提供更方便的介面)
     - 包含大部分 C Standard Library 函式 (stdio.h 、 stdlib.h)
     - 擴充系統相關功能

  3. Shell 和工具

  4. 還有很多

$ POSIX

分類	POSIX 規範下須提供的函式 (只列出這門課會教到的)
File and I/O	open(2), read(0), write(1), close(3) fopen(), fread(), fwrite(), fclose()
Process Control	fork(57), getpid(39), getppid(110), wait(61), exec(59)
IPC	mq_open(), mq_close(), mq_unlink(), mq_send(), mq_receive() shm_open(), mmap(9), ftruncate(77), munmap(11), shm_unlink()
Threaed management	pthread_self(), pthread_equal(), pthread_create(), pthread_exit() pthread_join()
Socket Programming	socket(41), bind(49), inet_pton(), htons(), recvfrom(45), sendto(44), close(3)
Signal	kill(62), alarm(37), signal(), sigaction(13), pause(34)
Spin Lock	pthread_spin_init(), pthread_spin_lock(), pthread_spin_unlock(), pthread_spin_trylock(), pthread_spin_destroy()

$ POSIX

分類	POSIX 規範下須提供的函式 (只列出這門課會教到的)
Mutex and Condition Variable	pthread_mutex_init(), pthread_mutex_destroy() pthread_mutex_lock(), pthread_mutex_unlock()
Semaphore	sem_init(), sem_destroy(), sem_getvalue(), sem_wait(), sem_post() sem_open(), sem_close(), sem_unlink()

$ Strace

- 攔截並顯示程式呼叫的系統呼叫（system calls）與訊號（signals）

```c
#include<stdio.h>

int main(){
        printf("Hello world\n");
}
```

```
g114056117@oslab:~/lab/SystemCallVsAPI$ strace -c ./helloworld
Hello World
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 27.42    0.000769         769         1           execve
 17.72    0.000497          62         8           mmap
 10.37    0.000291          97         3           fstat
  8.56    0.000240          80         3           brk
  5.10    0.000143          71         2           openat
  4.92    0.000138          46         3           mprotect
  3.89    0.000109          54         2           pread64
  3.67    0.000103          51         2           close
  2.89    0.000081          81         1         1 access
  2.25    0.000063          63         1           write
  1.96    0.000055          55         1           getrandom
  1.89    0.000053          53         1           read
  1.64    0.000046          46         1           munmap
  1.60    0.000045          45         1           set_tid_address
  1.60    0.000045          45         1           set_robust_list
  1.60    0.000045          45         1           rseq
  1.57    0.000044          44         1           arch_prctl
  1.35    0.000038          38         1           prlimit64
------ ----------- ----------- --------- --------- ----------------
100.00    0.002805          82        34         1 total
```

$ Permission Code

- 在類 Unix 系統中，權限碼（permission codes）用來控制誰可以讀取（read）、寫入（write）、或執行（execute）一個檔案或目錄。
- ls -l ：以長格式（detailed format）列出目前目錄中的檔案與目錄。

```
g114056117@oslab:~/lab/SystemCallVsAPI/example$ ls
fread_file    fwrite_file    hello.txt  read_file.c  write_file.c
fread_file.c  fwrite_file.c  read_file  write_file
g114056117@oslab:~/lab/SystemCallVsAPI/example$ ls -l
total 84
-rwxrwxr-x 1 g114056117 g114056117 16232 Aug 27 04:07 fread_file
-rw-rw-r-- 1 g114056117 g114056117   530 Aug 26 03:39 fread_file.c
-rwxrwxr-x 1 g114056117 g114056117 16192 Aug 27 04:07 fwrite_file
-rw-rw-r-- 1 g114056117 g114056117   907 Aug 27 04:05 fwrite_file.c
-rw-r--r-- 1 g114056117 g114056117    13 Sep 12 04:52 hello.txt
-rwxrwxr-x 1 g114056117 g114056117 16144 Sep 12 04:48 read_file
-rw-rw-r-- 1 g114056117 g114056117   723 Sep 12 04:50 read_file.c
-rwxrwxr-x 1 g114056117 g114056117 16104 Sep 12 04:52 write_file
-rw-rw-r-- 1 g114056117 g114056117   774 Sep 12 04:51 write_file.c
```

$ Permission Code

r (read) → 可讀取檔案
w (write) → 可修改檔案內容
x (execute) → 可執行

Octal notation

4 2 1 4 2 1 4 2 1 = 755

d r w x r w x r w x 2 root root bytes month date year path

Owner **Group** **Other**

Three Classes

File size

Last review time

File path

To indicate what type it is
- : not a directory
d: directory

$ Permission Code

```
g114056117@oslab:~/lab/SystemCallVsAPI/example$ ls -l hello.txt
-rw-r--r-- 1 g114056117 g114056117 13 Sep 12 04:52 hello.txt
g114056117@oslab:~/lab/SystemCallVsAPI/example$ cat hello.txt
Hello World
```

檔案擁有者權限 → 只寫

```
g114056117@oslab:~/lab/SystemCallVsAPI/example$ chmod 0244 hello.txt
g114056117@oslab:~/lab/SystemCallVsAPI/example$ ls -l hello.txt
--w-r--r-- 1 g114056117 g114056117 13 Sep 12 04:52 hello.txt
g114056117@oslab:~/lab/SystemCallVsAPI/example$ cat hello.txt
cat: hello.txt: Permission denied
```

$ system call I/O interface(1/4)

open()

```
#include <unistd.h> //Declares system calls like open(), read(), write(), close()
#include <fcntl.h> //Define flags
```

```
int open ( const char *filename, int flags, mode_t mode);
int open ( const char *filename, int flags);
```

```
EX: f1 = open ( "hello.txt", O_CREAT | O_WRONLY | O_TRUNC, 0644);
```

O_RDONLY	Read only.
O_WRONLY	Write only.
O_RDWR	Read and write.
O_CREAT	Create file if it doesn't exist.
O_TRUNC	If the file already exists and the open mode allows writing (i.e., is O_RDWR or O_WRONLY) it will be truncated to length 0.
O_APPEND	Append data at the end of the file.

return value: Return the file descriptor, or -1 is returned and ***errno*** is set appropriately.

$ system call I/O interface(2/4)

read()

```
#include<unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count);
```

```
EX: read(f1, buffer, 4);
```

- fd: The file descriptor to read from. This is usually obtained by open().
- buf: A pointer to a buffer where the read data will be stored.
- count: The maximum number of bytes to read into the buffer.
- return value:
  - Returns the number of bytes actually read (≥ 0).
  - Returns 0 if end-of-file (EOF) is reached.
  - Returns -1 on error, and errno is set accordingly.

$ system call I/O interface(3/4)

write()

```
#include<unistd.h>
```

```
ssize_t write(int fd, const void *buf, size_t count);
```

```
EX: write(f1, buffer, 4);
```

- fd: The file descriptor of where to write the input.
- buf: Pointer to the buffer that holds the data you want to write.
- count: The number of bytes to write from the buffer.
- return value:
  - Returns the number of bytes actually written (may be less than count).
  - Returns -1 if an error occurs, and sets errno.

$ system call I/O interface(4/4)

close()

```
#include<unistd.h>
```

```
int close(int fd);
```

```
EX: close(f1);
```

- fd: The file descriptor of the file you want to close.
- return value:
  - returns zero on success.
  - On error, -1 is returned, and errno is set appropriately.

$ Example1 – write_file.c

write_file.c
~/lab/SystemCallVsAPI/example

```c
#include <unistd.h>     // for open(), read(), write(), close()
#include <fcntl.h>      // for open() flags (O_RDWR, O_CREAT, etc.)

int main() {
    int fd;                                 // file descriptor
    char buffer[] = "Hello World\n";    // message to write into the file

    // Open (or create if not exist) the file "hello.txt"
    // O_CREAT -> create the file if it does not exist
    // O_RDWR  -> open for write only
    // 0 -> leading zero = octal notation, Owner(6) -> read + write
    fd = open("hello.txt", O_CREAT | O_WRONLY, 0644);

    // Write the contents of 'buffer' into the file
    // sizeof(buffer) includes the trailing '\0', so it writes 13 bytes
    write(fd, buffer, sizeof(buffer));

    // Close the file descriptor to free resources
    close(fd);
    return 0;
}
```

$ Example1 – read_file.c

Open | read_file.c ~/lab/SystemCallVsAPI/example | Save

```c
#include <stdio.h>      // for printf()
#include <unistd.h>     // for open(), read(), write(), close()
#include <fcntl.h>      // for open() flags (O_RDWR, O_CREAT, etc.)

int main() {
    int fd;                     // file descriptor
    char buffer[100];           // buffer to hold contents read from the file

    // Open the file "hello.txt" in read-only mode
    fd = open("hello.txt", O_RDONLY);

    // Read up to sizeof(buffer) (100) bytes from the file into buffer
    // read() returns the actual number of bytes read, or -1 on error
    read(fd, buffer, sizeof(buffer));

    // Print the contents of the buffer to stdout
    printf("%s", buffer);

    // Close the file descriptor
    close(fd);
    return 0;
}
```

$ example1

```
g114056117@oslab:~/lab/SystemCallVsAPI$ gcc write_file.c -o write_file
g114056117@oslab:~/lab/SystemCallVsAPI$ gcc read_file.c -o read_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ./write_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ./read_file
Hello World
```

$ example2

- 如何透過 system call 將資料輸出到螢幕上？

  在 UNIX 裡，每個程式一啟動， OS 就自動幫它打開三個檔案描述子（FD）：

  - 0 → 標準輸入（stdin)
  - 1 → 標準輸出（stdout)
  - 2 → 標準錯誤（stderr)

- 預設情況下， stdout (FD = 1) 會連到「你開啟的終端機」

$ example2

write_to_terminator.c
~/lab/SystemCallVsAPI/example

```c
#include <unistd.h>

int main(){
    char buffer[] = "Hello World\n"; //message to write into a file

    // File descriptor 1 = standard output (stdout).
    // By default it goes to the terminal
    write(1, buffer, sizeof(buffer));
    return 0;
}
```

```
g114056117@oslab:~/lab/SystemCallVsAPI/example$ ./write_to_terminator
Hello World
```

C Library I/O interface (1/4)

- **fopen()**

```
#include <stdio.h>
```

```
FILE *fopen ( const char *filename, const char *mode );
```

```
EX: FILE *f1;
      f1 = fopen ( "XXX.txt", "w" );
```

mode:

"r"	**read:** Open file for input operations. The file must exist.	"r+"	open for reading and writing, file must exist, not truncated
"w"	open for writing, file is truncated if it exists, created if it doesn't	"w+"	open for reading and writing, file is truncated if it exists, created if it doesn't
"a"	open for appending, writes go to the end of the file	"a+"	open for reading and writing, writes always go to the end of the file

Return value: Successful: the function returns a pointer to the object that controls the stream

$ C Library I/O interface (2/4)

- **fread()**

```
#include <stdio.h>
```

```
size_t fread ( void * ptr, size_t size, size_t count, FILE * stream );
```

```
EX: fread ( buffer, 1, sizeof(buffer), f1 );
```

- ptr: Pointer to the destination buffer.
- size: The size (in bytes) of one element you want to read.
- count: How many elements to read. fread will attempt to read up to count elements, each of size bytes.
- stream: Pointer to a FILE object that specifies an input stream.
- return value: The total number of elements successfully read is returned.

$ C Library I/O interface(3/4)

- **fwrite()**

```
#include <stdio.h>
```

```
size_t fwrite ( const void * ptr, size_t size, size_t count, FILE * stream );
```

```
EX: fwrite ( buffer, sizeof(char), 4, f1 );
```

- ptr: Pointer to the array of elements to be written, converted to a const void*.
- size: Size, in bytes, of each element to be written. size_t is an unsigned integral type.
- count: Number of elements, each one with a size of size bytes.
- stream: Pointer to a FILE object that specifies an input stream.
- return value: The total number of elements successfully written is returned

$ C Library I/O interface(4/4)

- **fclose()**

```
#include <stdio.h>
```

```
Int fclose ( FILE * stream);
```

```
EX: fclose(f1);
```

- stream: Pointer to a FILE object that specifies the stream to be closed.
- return value: If the stream is successfully closed, a zero value is returned. On failure, EOF is returned.

$ example3

Open | fwrite_file.c — ~/lab/SystemCallVsAPI/example | Save

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp;                           // file pointer
    char buffer[] = "Hello World\n";    // message to write into the file

    // Open (or create if not exist) the file "hello.txt"
    // "w+" -> open for reading and writing; create file if it does not exist
    //         and truncate it to zero length if it does
    fp = fopen("hello.txt", "w+");
    if (fp == NULL) {                   // check if fopen() failed
        perror("fopen");                // print error message to stderr
        exit(1);                        // exit with error code
    }

    // Write the contents of 'buffer' into the file
    // fwrite(ptr, size_of_each_element, number_of_elements, file_stream)
    fwrite(buffer, sizeof(buffer), 1, fp);

    // Close the file to flush the buffer and free resources
    fclose(fp);
    return 0;
}
```

$ example3

Open | fread_file.c ~/lab/SystemCallVsAPI | Save

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    FILE *fp;                   // file stream
    char buffer[100];           // buffer to hold file contents

    // Open the file "hello.txt" in read-only binary mode
    fp = fopen("hello.txt", "rb");
    if (fp == NULL) {
        perror("fopen");
        exit(1);
    }

    // Read sizeof(buffer) bytes
    size_t n = fread(buffer, 1, sizeof(buffer), fp);

    // Print the contents
    printf("%s", buffer);

    // Close the file
    fclose(fp);
    return 0;
}
```

$ example3

```
g114056117@oslab:~/lab/SystemCallVsAPI$ gcc fwrite_file.c -o fwrite_file
g114056117@oslab:~/lab/SystemCallVsAPI$ gcc fread_file.c -o fread_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ./fwrite_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ./fread_file
Hello World
```

$ example4 - read() vs fread()

Step 1: 創建一個 10MB 的 file

```
g114056117@oslab:~/lab/SystemCallVsAPI$ gcc create_file.c -o create_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ./create_file
g114056117@oslab:~/lab/SystemCallVsAPI$ ls -l my_file.txt
-rw-r--r-- 1 g114056117 g114056117 10485760 Sep 23 07:54 my_file.txt
```

$ example4 - read() vs fread()

Step 2: 分別用 read() 和 fread() 去讀取 my_file.txt ，一次只讀 1 個 byte ，讀 10M 次，比較讀取時間差異

read()

```
g114056117@oslab:~/lab/SystemCallVsAPI$ sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"
g114056117@oslab:~/lab/SystemCallVsAPI$ ./sequential_read 1
2.944057
```

fread()

```
g114056117@oslab:~/lab/SystemCallVsAPI$ sudo sh -c "echo 3 > /proc/sys/vm/drop_caches"
g114056117@oslab:~/lab/SystemCallVsAPI$ ./sequential_fread 1
0.138395
```

$ example4 - read() vs fread()

Step 3：用 strace 查看

read()

```
g114056117@oslab:~/lab/SystemCallVsAPI$ strace -c ./sequential_read 1
663.722539
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
100.00  106.215242          10  10485762           read
```

fread()

```
g114056117@oslab:~/lab/SystemCallVsAPI$ strace -c ./sequential_fread 1
1.567096
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 99.16    0.245695          95      2562           read
  0.15    0.000383          47         8           mmap
  0.13    0.000312         104         3           close
```

$ Exercise

- 學習目標：了解為何我們比較偏好使用將 system call 包裝後的函式
- 要求：在只使用 <unistd.h> 的情況下印出 integer
- 助教會檢查程式碼看各位有沒有偷用 <stdio.h>

```c
int main() {
    my_print(0);
    my_print(100);
    my_print(2147483647);
    my_print(-2147483648);
    return 0;
}
```

```
g114056117@oslab:~/lab/SystemCallVsAPI$ ./lab3
0
100
2147483647
-2147483648
```



Thanks for your listening

Made by PurpleRed in 2023/08